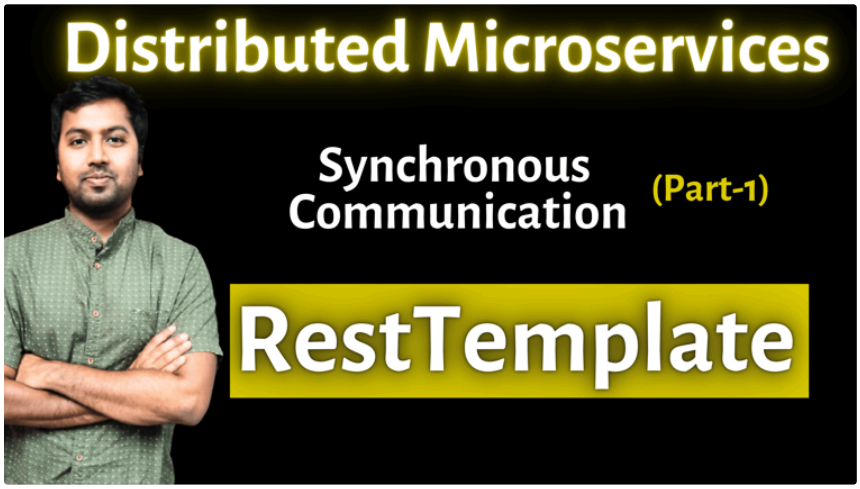
RestClient - Synchronous Communication

Prerequisite to understand RestClient is, to first understand its predecessor RestTemplate



Kindly check the Part-1 (RestTemplate), if there is any doubt with that topic.

<u>In previous video, we saw the Limitation of RestTemplate:</u>

Concept && Coding

- In RestTemplate, there are already so many overloaded methods, so its hard to remember and maintain.
- RestTemplate was build before concepts like Retry, circuit breaker etc.. So adding support means more overloaded methods and not user friendly.
- RestTemplate is in Maintenance mode - means no new feature, only bug fixes.

Alternative of RestTemplate: **<u>RestClient</u>**

- Introduced in Spring Framework 6.0+ and SpringBoot 3.0+
- Synchronous/blocking in nature, means client wait for the response from the Server before continuing.
- Modern, fluent based API that is more readable and easy to maintain.

OrderService needs to invoke ProductService

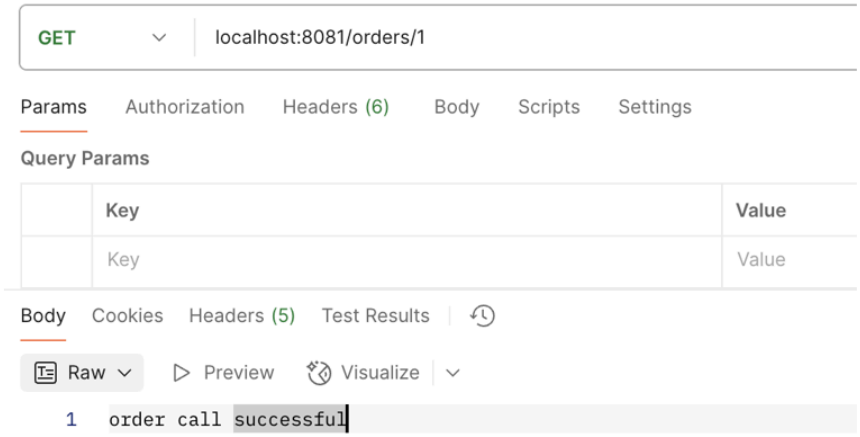
OrderService (port 8081) ----------⟶ **ProductService** (port 8082)

```
@Configuration
public class AppConfig {

    @Bean
    public RestClient restClientInstance() {
        return RestClient.create();
    }
}

```

```java
@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping("/{id}")
    public String getProduct(@PathVariable String id) {
        return "Product fetched with id: " + id;
    }
}

```

```java
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    RestClient restClient;

    @GetMapping("/{id}")
    public ResponseEntity<String> getOrder(@PathVariable String id) {

        String response = restClient
                .get()
                .uri("http://localhost:8082/products/" + id)
                .retrieve()
                .body(String.class);

        System.out.println(
                "Response from Product API called from order service: " + response
        );

        return ResponseEntity.ok("order call successful");
    }
}

```

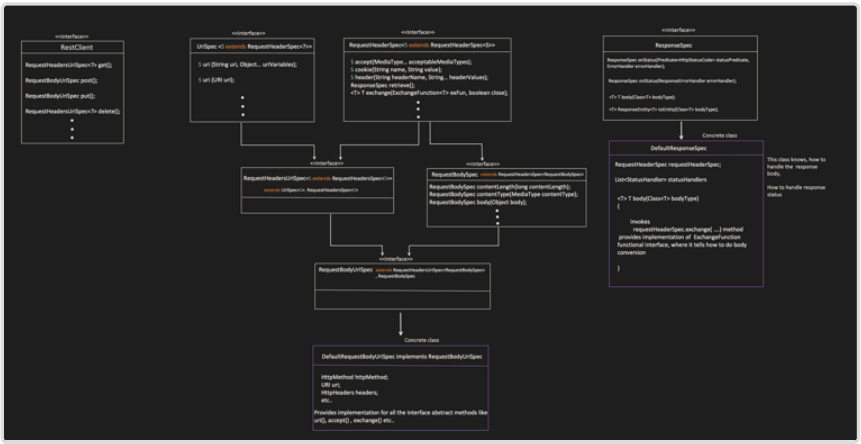
Concept && Coding



So, first important thing to understand is, how this Fluent API works?

- Fluent API means chaining of method calls.

- Each method call returns an object (of next step) that exposes next set of operations (methods).



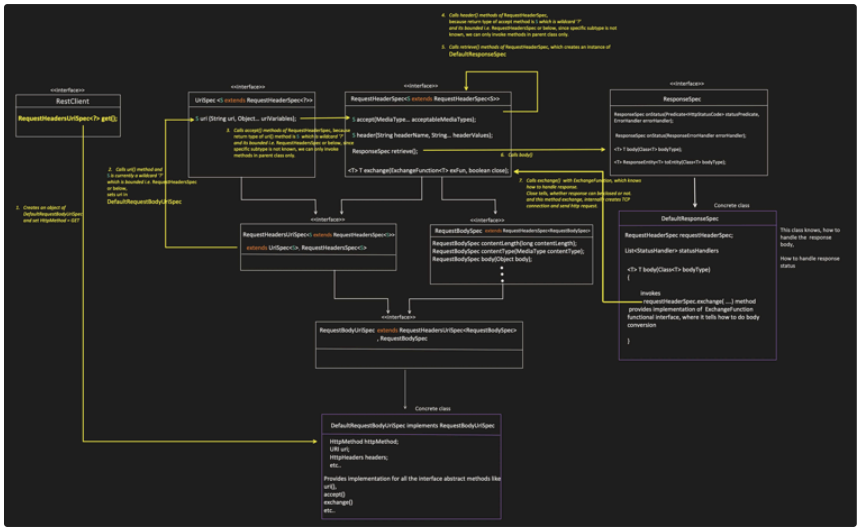
I am assuming that, we all know about Generics Class and Wildcards in Java

Now, lets see how Fluent API works for GET, POST, DELETE operation.

Concept && Coding

GET:

```
RestClient restClient = RestClient.create();

String response = restClient
        .get()
        .uri("http://localhost:8082/products/" + id)
        .accept(MediaType.APPLICATION_JSON)
        .header("X-Custom-Header", "xyz")
        .retrieve()
        .body(String.class);


```

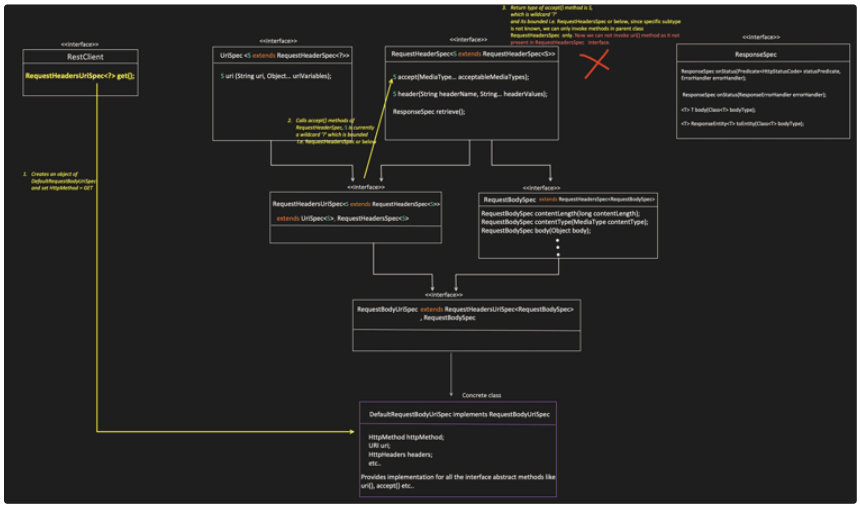


Let see, how flow can be broken

GET

```
RestClient restClient = RestClient.create();

String response = restClient
        .get()
        .accept(MediaType.APPLICATION_JSON)
                .uri
```

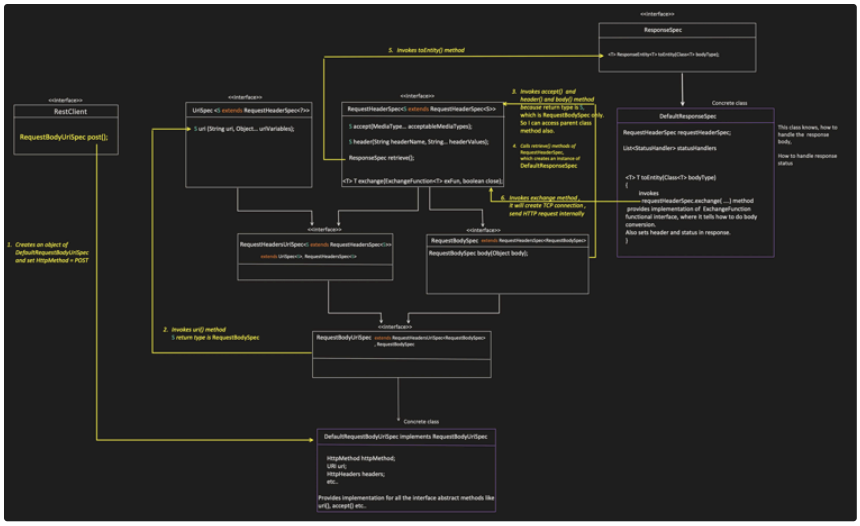
Instead of uri method first, I invoked the accept method first and now I can not set the URI itself. Why, does sequence of call matters?



Concept && Coding

POST:

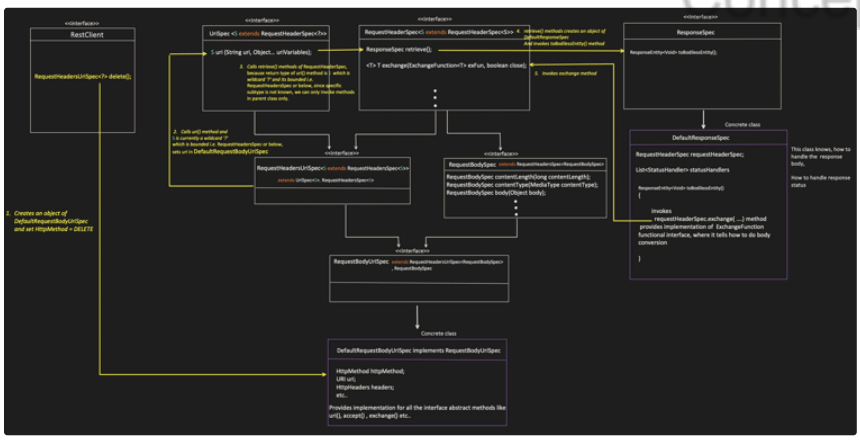
```
RestClient restClient = RestClient.create();

ResponseEntity<ProductEntity> response = restClient
        .post()
        .uri("http://localhost:8082/products/create")
        .accept(MediaType.APPLICATION_JSON)
        .header("Content-Type", "application/json")
        .body(new ProductEntity("Ice-cream")) // some new object which need to be created
        .retrieve()
        .toEntity(ProductEntity.class);

ProductEntity responseBody = response.getBody();

```



DELETE:

```
RestClient restClient = RestClient.create();

ResponseEntity<Void> response = restClient
        .delete()
        .uri("http://localhost:8082/products/" + id)
        .retrieve()
        .toBodilessEntity();

HttpStatusCode deletionStatus = response.getStatusCode();

```

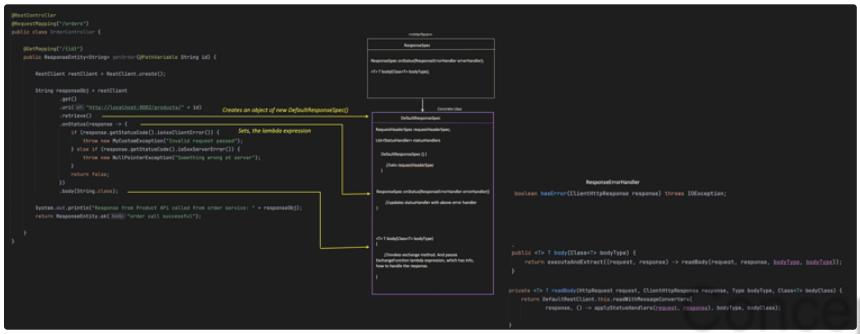
Concept && Coding



Exception Handling:

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @GetMapping("/{id}")
    public ResponseEntity<String> getOrder(@PathVariable String id) {

        RestClient restClient = RestClient.create();

        String responseObj = restClient
                .get()
                .uri("http://localhost:8082/products/" + id)
                .retrieve()
                .onStatus(response -> {
```

```java

                    if (response.getStatusCode().is4xxClientError()) {
                        throw new MyCustomException("Invalid request passed");
                    } else if (response.getStatusCode().is5xxServerError()) {
                        throw new NullPointerException("Something wrong at server");
                    }

                    return false;
                })
                .body(String.class);

        System.out.println(
                "Response from Product API called from order service: " + responseObj
        );

        return ResponseEntity.ok("order call successful");
    }
}

```



Concept && Coding

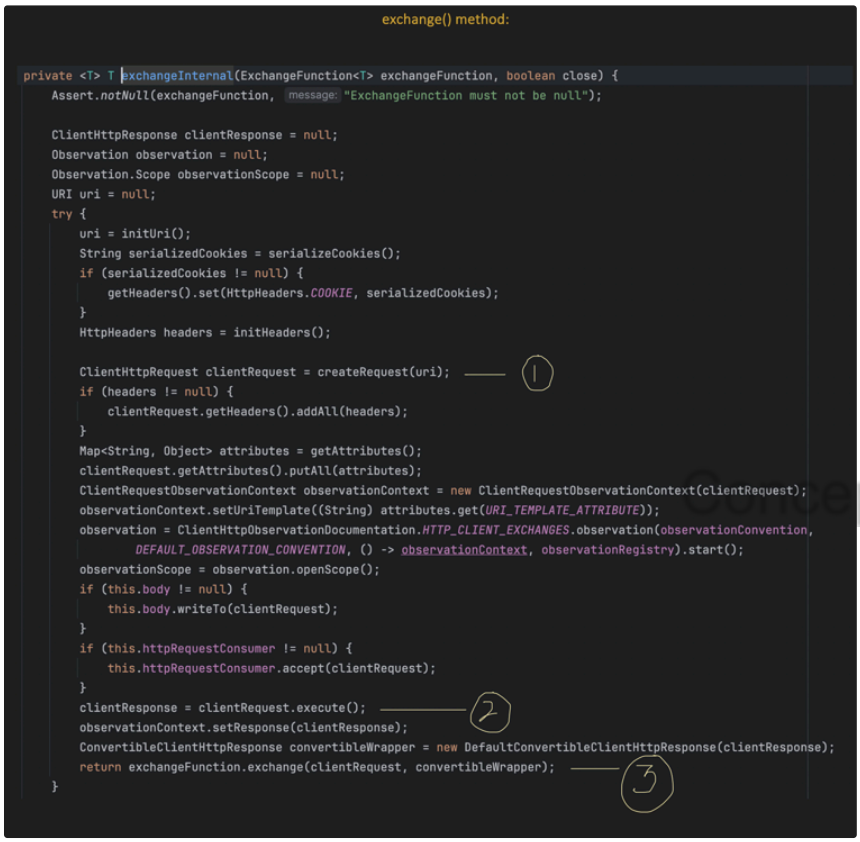
If we want full control over Response building and Exception handling

Then we can directly call exchange method, instead of relying of ResponseSpec to do that.

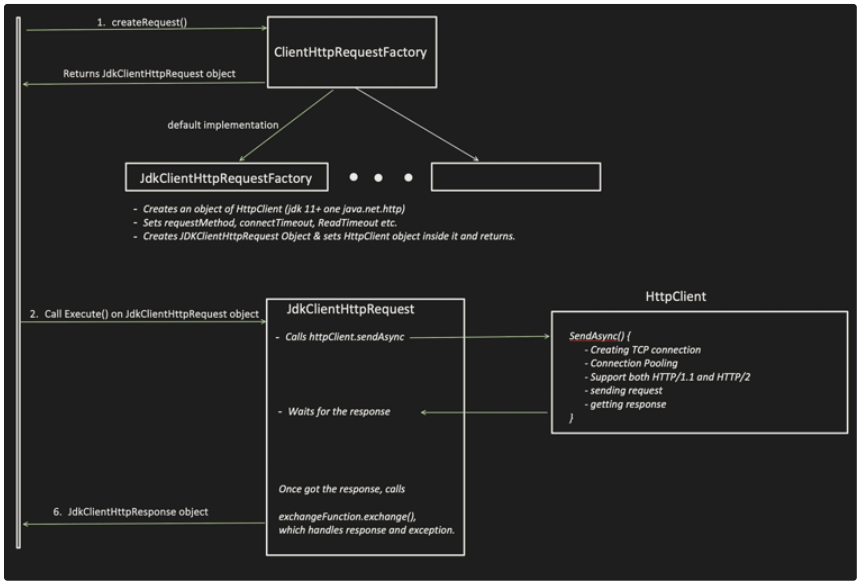
```java
@RestController
@RequestMapping("/orders")
public class OrderController {

    @GetMapping("/{id}")
    public ResponseEntity<String> getOrder(@PathVariable String id) {

        RestClient restClient = RestClient.create();

        String responseObj = restClient
                .get()
                .uri("http://localhost:8082/products/" + id)
                .exchange((request, response) -> {

                    if (response.getStatusCode().is4xxClientError()) {
                        throw new MyCustomException("Invalid request passed");
                    } else if (response.getStatusCode().is5xxServerError()) {
                        throw new InternalError("Something wrong at server");
                    } else {
                        // response mapping logic if no error
                        return StreamUtils.copyToString(
                                response.getBody(),
                                StandardCharsets.UTF_8
                        );
```

```
                    }
                });

        System.out.println(
                "Response from Product API called from order service: " + responseObj
        );

        return ResponseEntity.ok("order call successful");
    }
}

```

exchange() method:



Concept && Coding



Adding Interceptors

OrderService:

```java
public class MyCustomRequestInterceptor implements ClientHttpRequestInterceptor {

    @Override
    public ClientHttpResponse intercept(
            HttpRequest request,
            byte[] body,
            ClientHttpRequestExecution execution
    ) throws IOException {

        request.getHeaders().add("X-Custom-Header", "myvalue");
        return execution.execute(request, body);
    }
}


```

```java
@Configuration
public class AppConfig {

    @Bean
    public RestClient restClientInstance(
            ClientHttpRequestInterceptor myCustomInterceptor
    ) {
        return RestClient.builder()
                .requestInterceptor(myCustomInterceptor)
                .build();
    }

    @Bean
    public ClientHttpRequestInterceptor customRequestInterceptor() {
        return new MyCustomRequestInterceptor();
    }
}

```

Concept && Coding

```java
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    RestClient restClient;

    @GetMapping("/{id}")
    public ResponseEntity<String> getOrder(@PathVariable String id) {

        ResponseEntity<String> responseEntityObj = restClient
                .get()
                .uri("http://localhost:8082/products/" + id)
                .retrieve()
                .toEntity(String.class);

        System.out.println(
                "Response from Product API called from order service: "
                        + responseEntityObj.getBody()
        );

```

```
        return ResponseEntity.ok("order call successful");
    }
}

```

ProductService

```
@RestController
@RequestMapping("/products")
public class ProductController {

    @GetMapping("/{id}")
    public String getProduct(@PathVariable String id) {
        return "Product fetched with id: " + id;
    }
}

```

When product service got invoked, able to see the header which we added using interceptor

```
> this = {ProductController@5847}
> id = "1"
v headers = {LinkedHashMap@5845} size = 7
  > "connection" -> "Upgrade, HTTP2-Settings"
  > "content-length" -> "0"
  > "host" -> "localhost:8082"
  > "http2-settings" -> "AAEAAEAAAAIAAAABAAMAAABkAAQBAAAAAAUAAEAA"
  > "upgrade" -> "h2c"
  > "user-agent" -> "Java-http-client/17.0.12"
  > "x-custom-header" -> "myvalue"
```

Concept && Coding

Interceptor invokes at step2

```java
private <T> T exchangeInternal(ExchangeFunction<T> exchangeFunction, boolean close) {
    Assert.notNull(exchangeFunction, message: "ExchangeFunction must not be null");

    ClientHttpResponse clientResponse = null;
    Observation observation = null;
    Observation.Scope observationScope = null;
    URI uri = null;
    try {
        uri = initUri();
        String serializedCookies = serializeCookies();
        if (serializedCookies != null) {
            getHeaders().set(HttpHeaders.COOKIE, serializedCookies);
        }
        HttpHeaders headers = initHeaders();

        ClientHttpRequest clientRequest = createRequest(uri);
        if (headers != null) {
            clientRequest.getHeaders().addAll(headers);
        }
        Map<String, Object> attributes = getAttributes();
        clientRequest.getAttributes().putAll(attributes);
        ClientRequestObservationContext observationContext = new ClientRequestObservationContext(clientRequest);
        observationContext.setUriTemplate((String) attributes.get(URI_TEMPLATE_ATTRIBUTE));
        observation = ClientHttpObservationDocumentation.HTTP_CLIENT_EXCHANGES.observation(observationConvention,
                DEFAULT_OBSERVATION_CONVENTION, () -> observationContext, observationRegistry).start();
        observationScope = observation.openScope();
        if (this.body != null) {
            this.body.writeTo(clientRequest);
        }
        if (this.httpRequestConsumer != null) {
            this.httpRequestConsumer.accept(clientRequest);
        }
        clientResponse = clientRequest.execute();
        observationContext.setResponse(clientResponse);
        ConvertibleClientHttpResponse convertibleWrapper = new DefaultConvertibleClientHttpResponse(clientResponse);
        return exchangeFunction.exchange(clientRequest, convertibleWrapper);
    }
```

1

2

Interceptor, will get invoked when this method is trying to execute, our execute call is wrapped in InterceptingClientHttpRequest

3

Concept && Coding